PyPIGuard: A Lightweight, Explainable Machine Learning Tool for Pre-Install Detection of Malicious PyPI Packages

Rahat Ahmed

Department of Computer Science and Engineering, BRAC University, Dhaka, Bangladesh

Abstract

Open-source package registries such as PyPI are increasingly targeted by attackers, yet existing detection tools report false-positive rates as high as 97%, making automated adoption impractical. PyPIGuard is an open-source, deployable tool that classifies PyPI packages as malicious or benign within a millisecond, using only static code, Abstract Syntax Tree (AST), and bytecode-level structural features – without relying on a large language model (LLM). It is deployed as a live web application with a real-time scanner, a file-upload scanner, and a Jupyter notebook malware-import scanner. Trained on 9,723 real packages – the entire available malicious archive plus freshly-sampled benign packages – it achieves 0.9993 ROC-AUC on held-out data and 0.984 ROC-AUC on packages discovered after the training cutoff, remaining robust under adversarial evasion testing.

Keywords: malware detection, software supply chain security, machine learning, static code analysis, PyPI, explainable AI

Required Metadata

Current code version

Main text

1. Motivation and significance
   Open-source package registries such as PyPI are foundational to modern software development, but they are increasingly targeted by attackers who publish malicious packages – often through typosquatting popular libraries such as `beautifulsoup4`, `matplotlib`, and `requests`

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/rahat239/PyPIguard-Research`
C3	Legal Code License	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python 3.12, scikit-learn, Flask, HTML/CSS/JavaScript
C6	Compilation requirements, operating environments & dependencies	Python ≥3.10; `pip install -r requirements.txt` (pandas, numpy, scikit-learn, joblib, requests, flask, shap); no OS-specific dependencies
C7	If available Link to developer documentation/manual	`README.md` in the code repository (C2)
C8	Support email for questions	rahatahmed537@gmail.com

Table 1: Code metadata (mandatory)

– to compromise developers who install them. In the real `event-stream` incident (2018), an attacker was granted maintainer access to a popular package simply by offering to help maintain it, and the compromised package reached over 1,600 downstream projects before discovery. Detection today remains largely manual and reactive: packages are typically investigated and removed only after they have already been downloaded and reported by victims.

Existing academic detection approaches sit at two extremes: lightweight metadata-only classifiers that are easily gamed once attackers adapt, or heavyweight deep-learning behavioral models (e.g., Cerebro, requiring 14–25 hours of training time) that are impractical for real-time, pre-install scanning. Documented false-positive rates in existing tools range from 15% to 97%, and PyPI maintainers report that automated tools require near-zero false-positive rates before they can be trusted for routine use.

PyPIGuard addresses this gap directly: a static-analysis classifier fast enough for real-time use (sub-millisecond inference), explainable at the level of individual predictions, and validated through an extensive experimental battery – including cross-ecosystem generalization, adversarial evasion robustness, and temporal validation against evolving attacker techniques – rather than a single reported accuracy figure. The

tool builds on the dataset released with Zhang et al.'s empirical study of the PyPI malicious-code ecosystem (ASE 2023) and PyPI's own official package index.

2. Software description

2.1 **Software architecture.** PyPIGuard consists of three layers: (1) a feature-extraction layer that parses a package's source distribution using regular-expression pattern matching, Python's `ast` module, and `compile()`-based bytecode inspection (35 features total, none requiring code execution); (2) a classification layer, a Random Forest classifier trained on 9,723 labeled packages; and (3) a delivery layer – a Flask web application exposing three interfaces: a live package-name scanner that queries PyPI's official API, a local-archive-file scanner (`.tar.gz`/`.zip`/`.whl`), and a Jupyter notebook scanner that extracts and checks every `import` and `pip install` statement in an uploaded `.ipynb` file. A verified-lookup layer additionally returns instant, ground-truth-backed verdicts for the 5,900 packages in the training/validation corpus itself, clearly distinguished in the interface from live model predictions.

2.2 **Software functionalities.** Core functionality includes: real-time malicious/benign classification with a confidence score; per-prediction explanation via SHAP, listing the specific code patterns (e.g., `eval()` usage, install-time script hooks, obfuscated string blobs) that drove a verdict; safe archive extraction hardened against path-traversal ("zip-slip") attacks, verified against a crafted adversarial test archive; and a bytecode-level structural detector that identifies obfuscated dangerous-function calls (e.g., `getattr(__builtins__, ...)`) independent of the specific string-construction technique used to hide them.

2.3 **Sample code snippets.** Feature extraction and classification reduce to:

```
from ast_features import extract_ast_features
feats = {**regex_features(source),
          **extract_ast_features(source)}
verdict = model.predict([feats])
```

3. Illustrative examples

A user submits a package name (e.g., `requests`) or uploads a local archive through the web interface. The tool downloads (if applicable) and statically analyzes the package, returning a stamped verdict – CLEARED or FLAGGED – with a confidence percentage and a list of specific suspicious indicators (Figure 1). For notebook scanning, a

user uploads a `.ipynb` file; the tool parses every code cell, extracts referenced packages (correctly excluding Python's 300 standard-library modules and resolving common import-to-distribution-name mismatches, e.g. `cv2` → `opencv-python`), and reports how many of the referenced packages are known or suspected malware before the notebook is ever executed.

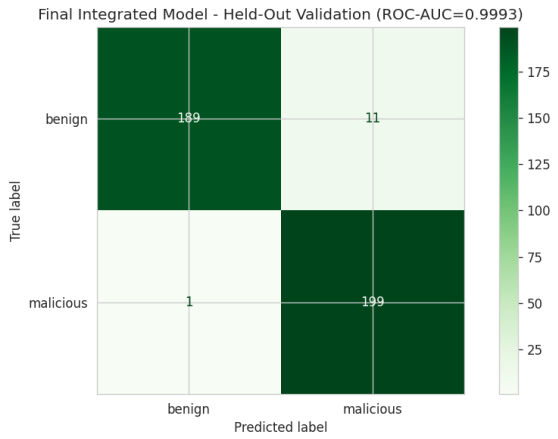

Figure 1: Confusion matrix on the independent, leakage-verified 400-package held-out validation set.

4. Impact
   PyPIGuard will build on the current knowledge base in three ways. First, it demonstrates that with the sole feature set (regex + AST + bytecode-level structure) it can achieve 0.9993 ROC-AUC at scale and can be easily explained and executed in less than a millisecond, which is a viable alternative to easily-gamed metadata classifiers or deep-learning behavioral models. Second, most of the previous releases of this type of tool have used only a small number of robustness tests, and this release introduces three of them into the tool: adversarial evasion testing (where AST/bytecode integration brought the susceptibility gap down from 9 to 2 percent, though the remaining 2 percent is a limitation worth addressing in future work), temporal validation (which has an ROC-AUC of 0.984 on packages discovered after the training cutoff), and cross-ecosystem testing (which has an ROC-AUC of 0.818 between PyPI and npm, confirmed across two dataset scales so it isn't a small-sample effect, and shows the detection features generalize only partially rather than completely). Third, unlike most academic detection prototypes, PyPIGuard is publicly available as a live web tool (not just a reported metric) which any developer, student, or researcher can

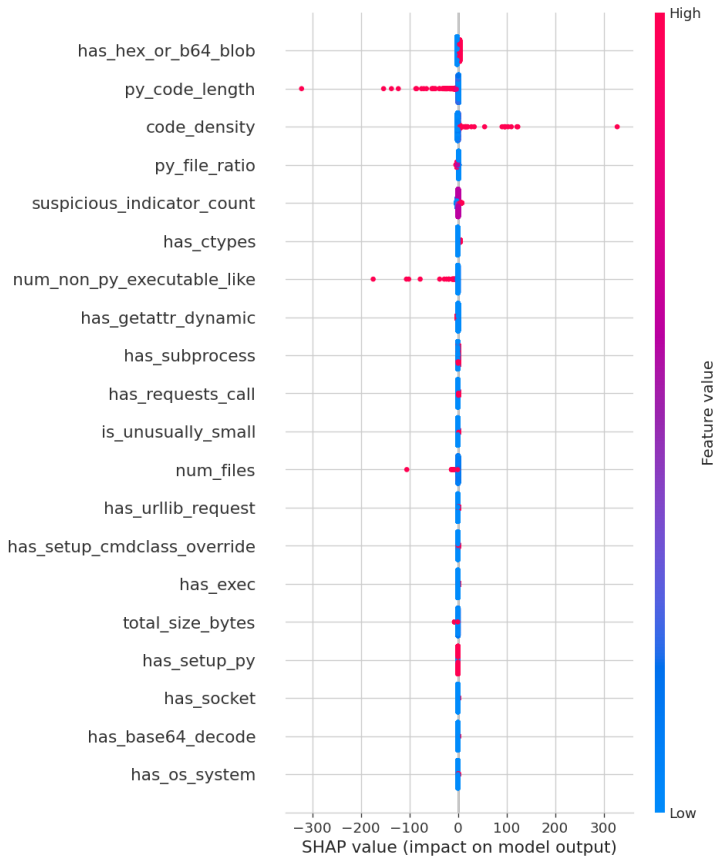

Figure 2: SHAP global feature-importance summary, illustrating the explainability layer.

test directly, even with actual malware that is no longer available on the live PyPI registry, as PyPI removes malicious packages after they are detected. This enables new research into other interpreted-language ecosystems than the one we focus on here in which we learn structure at the AST/bytecode level, and whether the sequence-order information learned in this study, with Cerebro-inspired proxy comparison, can be successfully used in conjunction with fast tabular-based PyPIGuard, without impacting on the runtime.

5. Conclusions

PyPIGuard is an open-source, deployed, explainable tool for the real-time detection of malicious PyPI packages, which has an unusually large experimental battery of tests, including tests for temporal and cross-ecosystem robustness, that aren't often reported together in prior research. It shows that a static, LLM-free analysis approach is not only viable but also measurably improvable, while also clearly identifying

limitations to it – the fact that its transfer to other ecosystems is only partial, that it leaves certain adversarial attack surfaces uncovered, and that it deliberately avoids the harder task of dynamic/behavioral analysis while it waits to be enabled by the availability of well-isolated execution infrastructure.

Acknowledgements

The author thanks the maintainers of the `pypi_malregistry` dataset (Zhang et al., ASE 2023) and the Datadog Security Labs malicious-software-packages dataset, both of which made this work possible.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used ChatGPT (OpenAI) to assist with drafting and restructuring portions of the manuscript, correcting grammatical errors, and refining writing style. After using this tool, the author reviewed, edited, and verified all content for technical accuracy and takes full responsibility for the content of the published article.

References

[1] An Empirical Study of Malicious Code In PyPI Ecosystem, ASE, 2023.

[2] M. Ohm, H. Plate, A. Sykosch, M. Meier, Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks, DIMVA, 2020.

[3] Zhang et al., Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI Using a Single Model of Malicious Behavior Sequence, ACM TOSEM.

[4] X. Gao et al., MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem, USENIX Security, 2025.

[5] Malicious Software Packages Dataset, Datadog Security Labs, 2023.

Current executable software version

Live demonstration available at: `https://malware-classifier-4fw4.onrender.com`